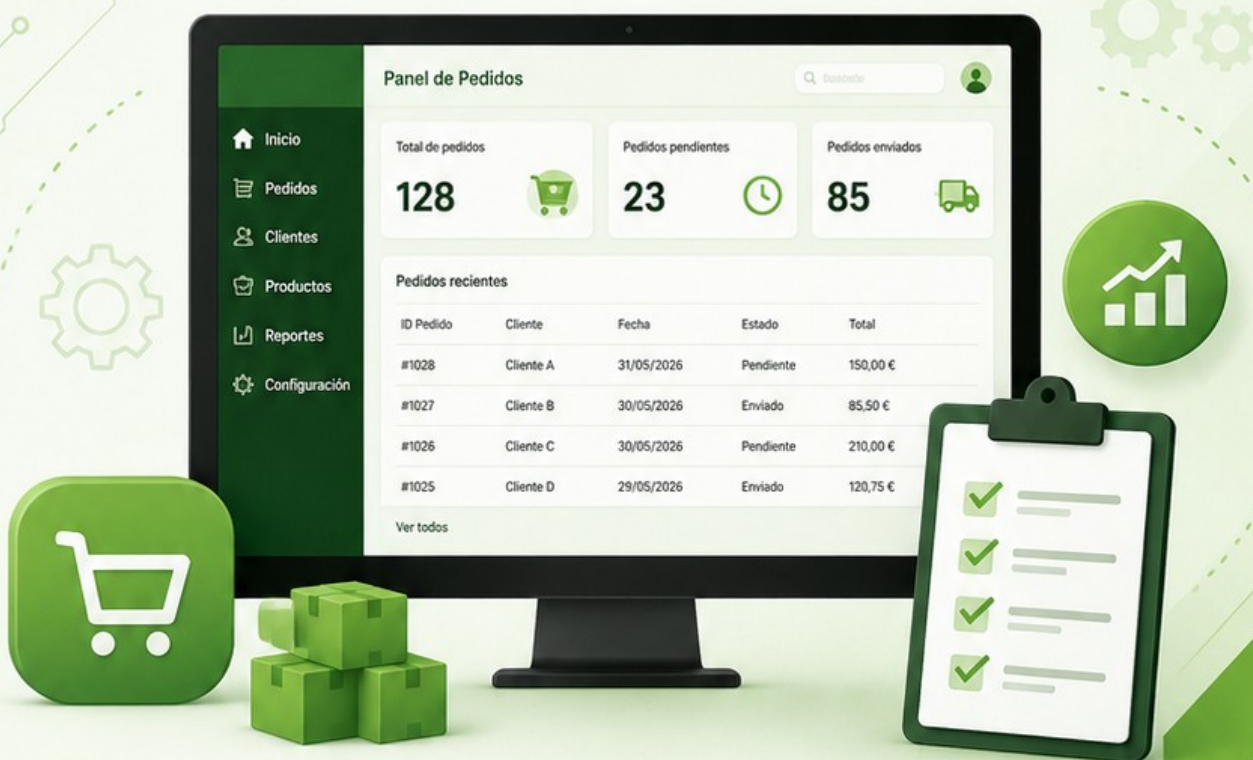


# 1ºDAW

# SISTEMA DE GESTIÓN DE PEDIDOS 1



**NOMBRE:**

Alexander Oviedo Cárdenas



**ASIGNATURA:**

Entornos de Desarrollo



**FECHA:**

31 de Mayo de 2026

```
<pedido>
  <id>1028</id>
  <cliente>Cliente A</cliente>
  <fecha>31/05/2026</fecha>
  <estado>Pendiente</estado>
  <total>150,00</total>
</pedido>
```



## 1. Introducción

En este proyecto va sobre el desarrollo de una aplicación orientada a objetos para la gestión de pedidos en un pequeño sistema comercial.

Objetivos del proyecto:

- Aplicar los principios fundamentales de la Programación Orientada a Objetos (POO)
- Diseñar y representar la estructura del sistema mediante diagramas UML
- Implementar el sistema en Java siguiendo el diseño previo
- Utilizar ingeniería inversa para comparar el diseño inicial con el código final

Alcance del sistema:

El sistema permite:

- Gestionar productos físicos y digitales
- Registrar clientes con su información básica
- Crear pedidos asociados a clientes
- Calcular automáticamente el precio final (IVA + costes de envío)
- Mostrar resúmenes detallados de los pedidos

## 2. Diseño del sistema

### 2.1 Decisiones de diseño

Se han tomado las siguientes decisiones arquitectónicas:

Producto como clase abstracta → Permite definir un comportamiento común y forzar la implementación de `calcularPrecioFinal` en las subclases.

Herencia para `ProductoFisico` y `ProductoDigital` → Cada tipo tiene reglas de negocio diferentes (IVA, coste de envío, licencias)

Composición en `Pedido` > `Producto` → Un pedido tiene productos; si el pedido desaparece, los productos asociados también (en el contexto del pedido)

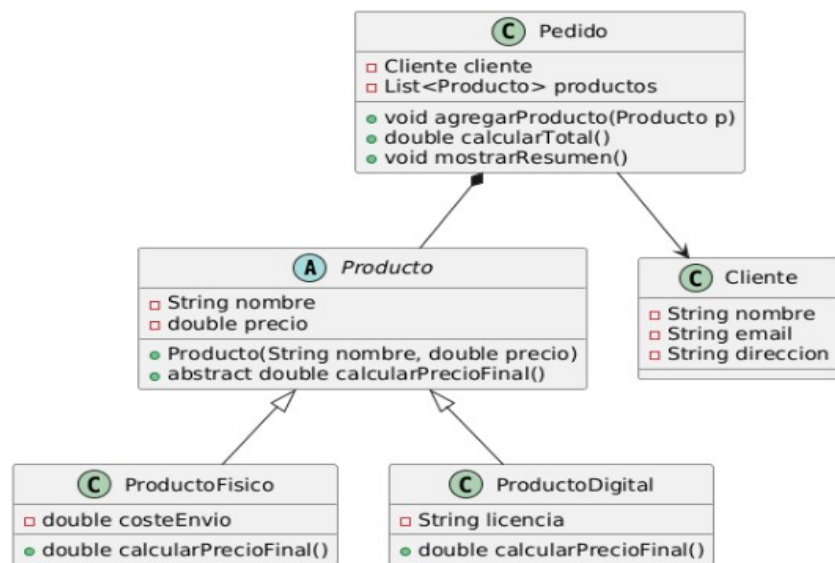
Asociación `Pedido` > `Cliente` → Un pedido pertenece a un cliente, pero el cliente puede existir sin pedidos

Encapsulación (atributos privados + getters/setters) → Protege los datos y permite controlar el acceso/modificación

### 2.2 Estructura de clases

| Clase           | Tipo          | Atributos principales                      | Métodos clave                                             |
|-----------------|---------------|--------------------------------------------|-----------------------------------------------------------|
| Producto        | Abstracta     | nombre, precio                             | <code>calcularPrecioFinal</code> y <code>toString</code>  |
| ProductoFisico  | Subclase      | <code>costeEnvio</code>                    | <code>calcularPrecioFinal</code> (IVA 21% + envío)        |
| ProductoDigital | Subclase      | licencia                                   | <code>calcularPrecioFinal</code> (IVA 10%)                |
| Cliente         | Independiente | nombre, email, direccion                   | Getters/setters y <code>toString</code>                   |
| Pedido          | Contenedora   | cliente, <code>List&lt;Producto&gt;</code> | <code>AgregarProducto</code> y <code>calcularTotal</code> |

## 2.3 Relaciones entre clases

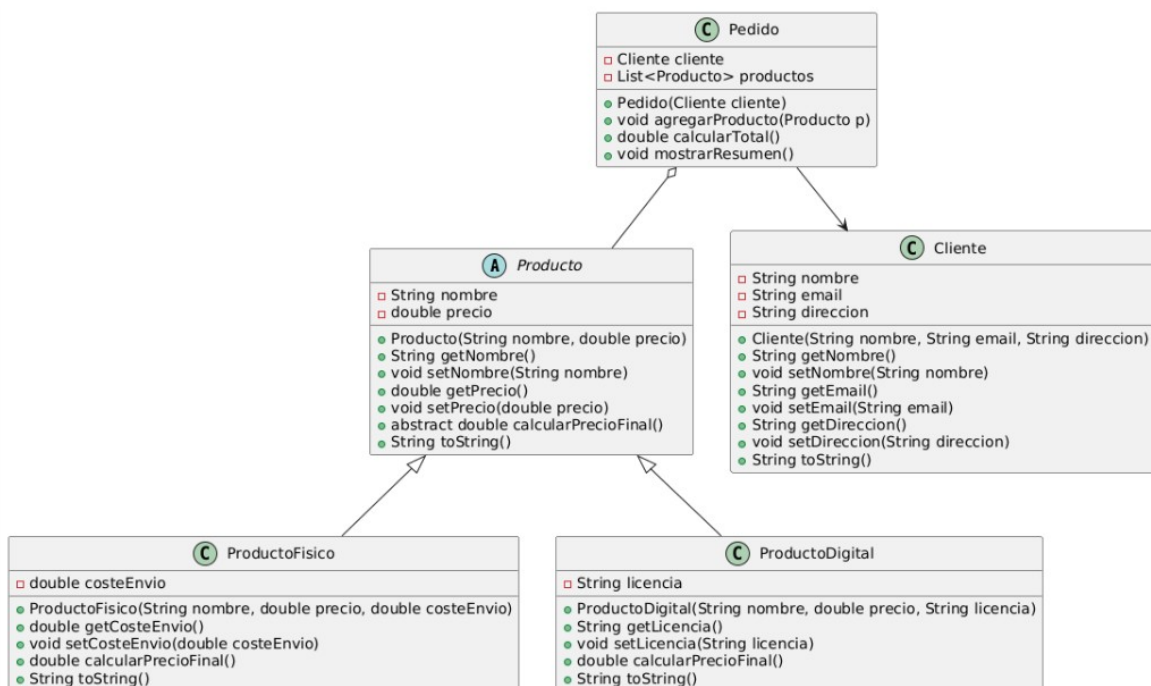


Leyenda de relaciones:

- Herencia (|-->): **ProductoFisico** y **ProductoDigital** heredan de **Producto**
- Composición (o-->): **Pedido** compone **Producto** (relación fuerte)
- Asociación (-->): **Pedido** se asocia con **Cliente**

## 3. Diagramas UML

### 3.1 Diagrama de clases inicial (ingeniería directa)



### 3.3 Comparativa entre diagramas

| Aspecto    | Diagrama inicial                  | Diagrama final                     | Diferencia |
|------------|-----------------------------------|------------------------------------|------------|
| Clases     | 5 clases                          | 5 clases                           | Coinciden  |
| Atributos  | Los especificados                 | Mismos getters/setters             | Coinciden  |
| Métodos    | Solo los principales              | Se añadieron todos getters/setters | Mejora     |
| Relaciones | Herencia, composición, asociación | Mismas relaciones                  | Coinciden  |

#### Conclusiones

- El diseño inicial fue fiel a la implementación final
- La ingeniería directa permitió tener una guía clara antes de programar
- Durante la implementación se añadieron métodos auxiliares (getters/setters) que no estaban en el diseño inicial pero que no alteran la estructura
- No hubo necesidad de modificar las relaciones planteadas inicialmente

## 4. Implementación en código

### 4.1 Estructura del proyecto

Proyecto/

```
|— src/  
|   |— modelo/  
|   |   |— Producto.java  
|   |   |— ProductoFisico.java  
|   |   |— ProductoDigital.java  
|   |   |— Cliente.java  
|   |   |— Pedido.java  
|   |— app/  
|   |— Main.java  
|— diagrams/  
|   |— diagrama-inicial.puml  
|   |— diagrama-inicial.png  
|   |— diagrama-final.puml  
|   |— diagrama-final.png  
|— README.md  
|— Gestion_pedidos_1.pdf
```

## 5. Conclusiones

Logros alcanzados

Se aplicaron correctamente los cuatro pilares de la POO:

Abstracción: Clase Producto abstracta

Encapsulación: Atributos privados + getters/setters

Herencia: ProductoFisico y ProductoDigital extienden Producto

Polimorfismo: calcularPrecioFinal() con comportamientos diferentes

Se completaron ambas fases de modelado UML:

Ingeniería directa (diseño → código)

Ingeniería inversa (código → diagrama)

La comparación entre diagramas demostró que el diseño inicial fue adecuado

Dificultades encontradas

Decisión sobre composición vs agregación: Se optó por composición porque los productos dentro de un pedido no tienen sentido fuera de él en el contexto del sistema.

Cálculo de IVA: Se decidió simplificar con porcentajes fijos (21% físicos, 10% digitales) para cumplir con el enunciado.

Posibles mejoras

| Mejora                | Descripción                                                |
|-----------------------|------------------------------------------------------------|
| Persistencia de datos | Guardar pedidos en base de datos o archivos                |
| Interfaz gráfica      | Añadir GUI con Swing o JavaFX                              |
| Descuentos            | Implementar códigos promocionales o descuentos por volumen |
| Inventario            | Controlar stock de productos físicos                       |
| Facturación           | Generar facturas en PDF                                    |
| Validaciones          | Controlar entradas (precios negativos, emails inválidos)   |